

stacked

Stacked cubes and dices

v0.1.0

Akilon <<https://github.com/Akilon27>>

MIT

A propos

Les empilements de cubes sont apparus dans les programmes de collège en 2025, d'où ce paquet.

Je me suis inspiré de `tikz3d-fr` (auteur : Cédric Pierquet) et d'une partie de `ProfCollege` de LaTeX (auteur : Christophe Poulain) mais l'implémentation n'en est pas une copie. J'ai utilisé l'IA (Gemini et Claude) pour les dés et la partie aléatoire de la fonction `pile`, le reste est de moi sauf la fonction `dice-face` créée par `eric1102` (sur Discord), incluse ici avec son accord.

Fonctions : « piles » de cubes et dés

- `cube()`
- `pile()`
- `building()`
- `above-view()`
- `right-view()`
- `front-view()`
- `below-view()`
- `left-view()`
- `back-view()`
- `thetaphi()`
- `dice()`
- `throws()`
- `dice-face()`

cube

Commande `cetz` pour un cube

Paramètres

```
cube(  
  a: array ,  
  c: auto color array ,  
  border-stroke: boolean color ,  
  thickness: int float ratio ,  
  light: float ratio array ,  
  side: boolean  
) -> function
```

a array

Position

c auto or color or array

auto → (rouge à droite, vert en haut, jaune à gauche), sinon une ou 3 couleurs

Par défaut: **auto**

border-stroke `boolean` or `color`

Lignes en noir ou de la couleur du cube, défaut false

Par défaut: `false`

thickness `int` or `float` or `ratio`

Epaisseur des tracés (de 1pt), défaut 100%

Par défaut: `100%`

light `float` or `ratio` or `array`

De combien éclaircir quand une seule couleur 20% par défaut

Par défaut: `20%`

side `boolean`

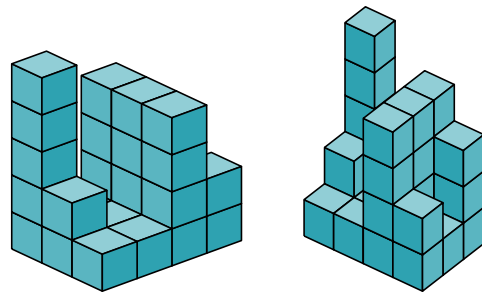
Commande interne pour la vue de côté avec pile

Par défaut: `false`

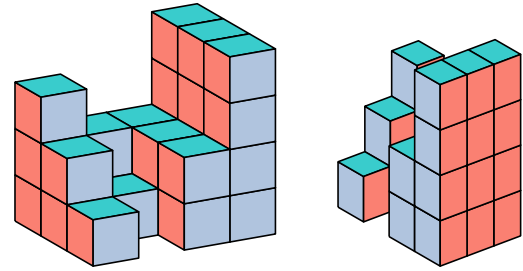
pile

Empilement de petits cubes sur une grille, donnés sous forme de liste allant du fond vers l'avant 3251 puis de gauche à droite (séparés par des virgules), une couleur peut être indiquée au début ou à la fin, couleurs de base avec none : avant → jaune, dessus → vert et droite → rouge, sinon une liste de 3 couleurs peut-être donnée dans colors

```
#pile(3415,1412,2411,eastern,side:true,  
iso:1,draw-scale:.6,border-stroke:true)
```

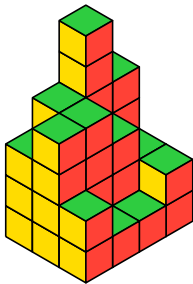


```
#pile(4223,4212,4201,colors:
(rgb("#fa8072"),rgb("#b0c4de"),teal),
side: true,iso: 2,draw-scale: .75)
```

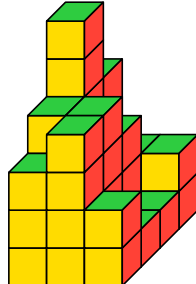


```
#grid(columns: 5*(1fr,), column-gutter: 1.5em,
..for i in (true,false,1,2,3) {[=== iso: #i
#pile("4643",2344,2112, iso: i)
],)}
)
```

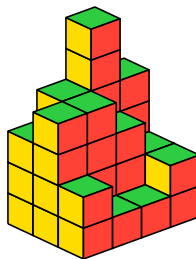
iso: true



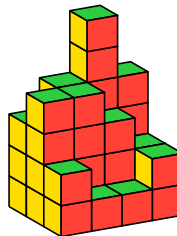
iso: false



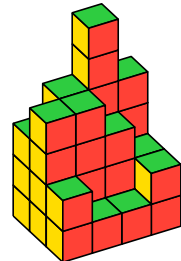
iso: 1



iso: 2

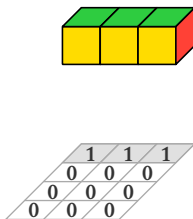


iso: 3

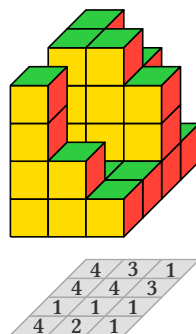


```
#grid(columns: 5*(3.2cm,), column-gutter: 2em,
..for i in (1,2,3,4) {[=== random-mode: #i
#pile(random:true, random-mode: i,shadow:true,seed:2*i,plan:true)
],)}
)
```

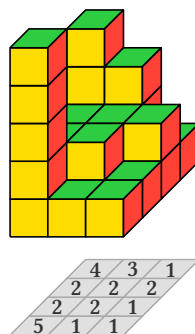
random-mode: 1



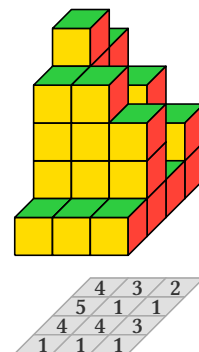
random-mode: 2



random-mode: 3



random-mode: 4



z:0 pour vue de face y:0, z:(0,-1) pour vue du dessus x:0, z:(-1,0) pour vue de droite

Paramètres

```
pile(  
  ..a: int str array arguments ,  
  random: boolean ,  
  width: int ,  
  depth: int ,  
  height: int ,  
  seed: int ,  
  plan: boolean ,  
  labels: boolean ,  
  x: array ,  
  y: array ,  
  z: array ,  
  thickness: int float ratio ,  
  border-stroke: boolean color ,  
  side: boolean ,  
  light: float ratio array ,  
  draw-scale: float ratio ,  
  x-spread: float int ,  
  y-spread: float int ,  
  z-spread: float int ,  
  colors: array ,  
  grids: boolean ,  
  solution: boolean ,  
  below-separation: auto int float ,  
  left-separation: auto int float ,  
  back-separation: auto int float ,  
  iso: boolean int ,  
  space: length relative length fraction ,  
  holes: boolean ,  
  random-mode: auto int ,  
  views: array ,  
  views-names: array ,  
  axes: boolean ,  
  opacity: ratio ,  
  shadow: boolean ,  
  disposition: str ,  
  box-args: dictionary  
) -> content
```

..a int or str or array or arguments

Liste de nombres/string correspondants aux hauteurs de l'arrière vers l'avant, de gauche à droite

random boolean

Active un mode aléatoire : génère automatiquement l'empilement (au lieu des arguments positionnels), avec des hauteurs égales ou décroissantes de rang en rang

Par défaut: false

width `int`

Largeur (nombre de tranches selon X) en mode aléatoire

Par défaut: `3`

depth `int`

Profondeur (nombre de chiffres par tranche selon Z) en mode aléatoire

Par défaut: `4`

height `int`

Hauteur maximale (selon Y) en mode aléatoire

Par défaut: `5`

seed `int`

Graine du générateur pseudo-aléatoire pour la reproductibilité du tirage : même graine → même empilement, à changer pour en obtenir un autre

Par défaut: `1`

plan `boolean`

Affiche le plan de niveau (vue de dessus chiffrée)

Par défaut: `false`

labels `boolean`

Affiche les intitulés des vues sur les grids

Par défaut: `false`

x `array`

Direction de la première coordonnée, défaut (1,0)

Par défaut: `(1,0)`

y `array`

Direction de la seconde coordonnée, défaut (0,1)

Par défaut: `(0,1)`

z array

Direction de la troisième coordonnée, défaut (-0.5,-0.5)

Par défaut: (-.5, -.5)

thickness int or float or ratio

Epaisseur des tracés (de 1pt), défaut 60% soit 0.6pt

Par défaut: 60%

border-stroke boolean or color

Lignes en noir, défaut false

Par défaut: false

side boolean

Ajout d'une seconde vue de l'assemblage depuis la « droite »

Par défaut: false

light float or ratio or array

De combien éclaircir quand une seule couleur

Par défaut: 20%

draw-scale float or ratio

Echelle du dessin : modifie la taille des arêtes d'un cube de base

Par défaut: .5

x-spread float or int

Permet « d'éclater » l'empilement construit vers la droite afin de mieux permettre une visualisation par « tranches »

Par défaut: 0

y-spread float or int

Permet « d'éclater » l'empilement construit vers le haut afin de mieux permettre une visualisation par « couches »

Par défaut: 0

z-spread float or int

Permet « d'éclater » l'empilement construit vers l'avant afin de mieux permettre une visualisation par « tranches »

Par défaut: 0

colors array

Pour donner une liste de 3 couleurs pour les trois faces (gauche, droite, haut)

Par défaut: ()

grids boolean

Pour afficher trois grids permettant à l'élève de dessiner directement les vues de face, de dessus et de gauche

Par défaut: false

solution boolean

Affiche, lorsqu'elle est positionnée à true, les vues de face, de dessus et de gauche du solide associé sur les grids

Par défaut: false

below-separation auto or int or float

Écart de la grille du dessous (auto = calculé automatiquement)

Par défaut: auto

left-separation auto or int or float

Écart de la grille de gauche (auto = calculé automatiquement)

Par défaut: auto

back-separation auto or int or float

Écart de la grille du fond (auto = calculé automatiquement)

Par défaut: auto

iso `boolean` or `int`

Pour avoir différentes perspectives :

- si true, représentation isométrique,
- si 1 vue tikz3d-fr,
- si 2 première vue de ProfCollege,
- si 3 seconde vue de Profcollege,
- sinon vue par défaut de cetz.

Par défaut: `false`

space `length` or `relative length` or `fraction`

Espacement avec la seconde vue

Par défaut: `2em`

holes `boolean`

Autorise des « trous » (hauteur nulle) dans l'empilement aléatoire à partir de la deuxième pile ; la première pile reste toujours « pleine » (hauteur ≥ 1 partout)

Par défaut: `false`

random-mode `auto` or `int`

Mode aléatoire spécifique :

- 1 (LCG double min par gemini),
- 2 et 4 (LCG décroissant inspiré de ProfCollege par Claude),
- 3 (package suiji)

Par défaut: `auto`

views `array`

Liste des vues à afficher parmi (« dessus », « face », « droite »)

Par défaut: (`"dessus"`, `"face"`, `"droite"`)

views-names `array`

Noms des vues affichées parmi (« dessus », « face », « droite »)

Par défaut: (`"Vue de dessus"`, `"Vue de face"`, `"Vue de droite"`)

axes `boolean`

Affiche les repères d'axes (coordonnées) au sol

Par défaut: `false`

opacity `ratio`

Opacité des faces des cubes (100% = opaque, 50% = semi-transparent)

Par défaut: `100%`

shadow `boolean`

Dessine l'empreinte au sol sous l'empilement

Par défaut: `false`

disposition `str`

Disposition des grids 2D : « 3d » (projetées autour) ou « cote-a-cote » (à côté de la 3D)

Par défaut: `"3d"`

box-args `dictionary`

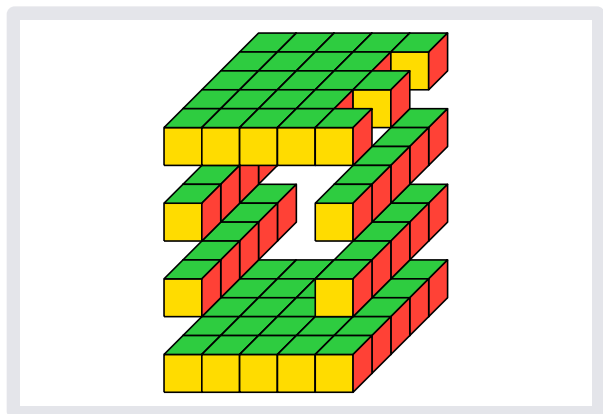
Arguments à passer à la (aux) boîte(s)

Par défaut: `()`

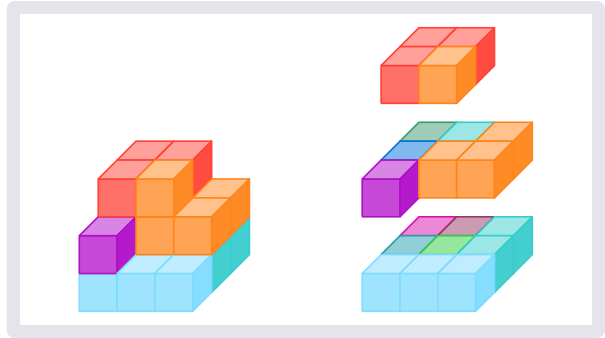
building

Assemblage de petits cubes avec mix de couleurs : chaque n° correspond à la position dans la liste de couleurs (16 max : 1 à 9 et a à g + t -> 3 couleurs), le « 0 » et le « o » représentent un trou, le « x » donne la première couleur, les cubes sont donnés dans une liste de listes : de l'avant vers l'arrière par un nombre/string « 5432a » puis étage par étage btt, listes de gauches à droite

```
#building((
  ("ttttt", "ttttt", "ttttt", "ttttt"),
  // Tranche la plus à gauche de bas en
  haut
  ("ttttt", "ooooo", "ooooo", "ttttt"),
  ("ttttt", "ooooo", "ooooo", "ttttt"),
  ("ttttt", "ooooo", "ooooo", "ttttt"),
  ("ttttt", "ttttt", "ttttt", "totot")
  // Tranche la plus à droite de même
), y-spread:1)
```



```
#building(((578,"21b",44), ("59a",63,64),
(533,66,0)),)
#h(3em)
#building(((578,"21b",44), ("59a",63,64),
(533,66,0)),,
y-spread:1.5)
```



z:0 pour vue de face y:0, z:(0,-1) pour vue du dessus x:0, z:(-1,0) pour vue de droite

Paramètres

```
building(
  ..a: array arguments ,
  color-list: array ,
  x: array ,
  y: array ,
  z: array ,
  thickness: int float ratio ,
  border-stroke: boolean color ,
  _rev,
  _dessous,
  _back,
  light: float ratio array ,
  draw-scale: float ratio ,
  iso: boolean int ,
  text: str content ,
  x-spread: float int ,
  y-spread: float int ,
  z-spread: float int ,
  box-args: dictionary
) -> content
```

..a array or arguments

Liste de nombres/string correspondants aux couleurs de l'arrière vers l'avant, de bas en haut, de gauche à droite

color-list array

Liste de max 16 couleurs

Par défaut: typst16

x array

Direction de la première coordonnée, défaut (1,0)

Par défaut: (1,0)

y `array`

Direction de la seconde coordonnée, défaut (0,1)

Par défaut: `(0,1)`

z `array`

Direction de la troisième coordonnée, défaut (-0.5,-0.5)

Par défaut: `(-.5,-.5)`

thickness `int` or `float` or `ratio`

Epaisseur des tracés (de 1pt), défaut 60% soit 0.6pt

Par défaut: `60%`

border-stroke `boolean` or `color`

Lignes en noir, défaut false

Par défaut: `false`

light `float` or `ratio` or `array`

De combien éclaircir quand une seule couleur

Par défaut: `20%`

draw-scale `float` or `ratio`

Echelle du dessin : modifie la taille des arêtes d'un cube de base

Par défaut: `.5`

iso `boolean` or `int`

Pour avoir différentes perspectives :

- si true, représentation isométrique,
- si 1 vue tikz3d-fr,
- si 2 première vue de ProfCollege,
- si 3 seconde vue de Profcollege,
- sinon vue par défaut de cetz

Par défaut: `false`

text `str` or `content`

Texte à mettre en dessous de l'empilement (pour les vues)

Par défaut: `none`

x-spread `float` or `int`

Permet « d'écarter » l'empilement construit vers la droite afin de mieux permettre une visualisation par « tranches »

Par défaut: `0`

y-spread `float` or `int`

Permet « d'écarter » l'empilement construit vers le haut afin de mieux permettre une visualisation par « couches »

Par défaut: `0`

z-spread `float` or `int`

Permet « d'écarter » l'empilement construit vers l'avant afin de mieux permettre une visualisation par « tranches »

Par défaut: `0`

box-args `dictionary`

Arguments à passer à la boîte

Par défaut: `()`

above-view

Vue du dessus pour une building

Paramètres

```
above-view(  
  ..it: arguments ,  
  text: str content  
) -> content
```

..it `arguments`

Arguments pour building

text `str` or `content`

Nom de la vue

Par défaut: "Vue du dessus"

right-view

Vue de droite pour une building

Paramètres

```
right-view(  
  ..it: arguments ,  
  text: str content  
) -> content
```

..it arguments

Arguments pour building

text `str` or `content`

Nom de la vue

Par défaut: "Vue de droite"

front-view

Vue de face pour une building

Paramètres

```
front-view(  
  ..it: arguments ,  
  text: str content  
) -> content
```

..it arguments

Arguments pour building

text `str` or `content`

Nom de la vue

Par défaut: "Vue de face"

below-view

Vue du dessous pour une building

Paramètres

```
below-view(  
  ..it: arguments ,  
  text: str content  
) -> content
```

..it arguments

Arguments pour building

text str or content

Nom de la vue

Par défaut: "Vue du dessous"

left-view

Vue de gauche pour une building

Paramètres

```
left-view(  
  ..it: arguments ,  
  text: str content  
) -> content
```

..it arguments

Arguments pour building

text str or content

Nom de la vue

Par défaut: "Vue de gauche"

back-view

Vue de l'arrière pour une building

Paramètres

```
back-view(  
  ..it: arguments ,  
  text: str content  
) -> content
```

..it arguments

Arguments pour building

text str or content

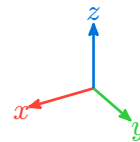
Nom de la vue

Par défaut: "Vue arrière"

thetaphi

Fonction qui détermine les définitions pour cetz.canvas de x, y et z pour un angle de vue défini par l'élévation (θ) et l'azimut (φ). Il faut la passer directement au canvas, sans oublier les deux points

```
#canvas(..thetaphi(theta:60, phi:150), {  
    import draw:*  
    repere  
})
```



Paramètres

```
thetaphi(  
    theta: int angle,  
    phi: int angle  
) -> dictionary
```

theta int or angle

Elévation θ

Par défaut: 70

phi int or angle

Azimut φ

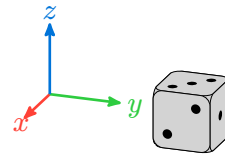
Par défaut: 110

dice

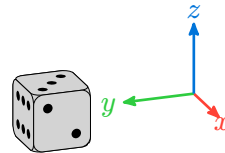
Fonction utilisée pour dessiner le dé en 3D.

Attention, l'utilisation de dice dans un canvas modifie la matrice !

```
#canvas({
  import draw:*
  dice(standalone:false, origin: (0,2),
    front:2, top:3, s:.8, vue: "D")
  repere
})
```



```
#canvas({
  import draw:*
  dice(standalone: false, origin: (0,2),
    front: 2, top:3, s:.8, vue: "G")
  repere
})
```



Paramètres

```
dice(
  standalone: boolean,
  front: int,
  top: int,
  s: int float ratio,
  theta: int float,
  phi: int float,
  vue: str,
  color: color,
  dot-color: color,
  origin: array,
  box-args: dictionary,
  opacity: ratio
) -> content function
```

standalone `boolean`

true si seul, false si déjà dans un canvas

Par défaut: `true`

front `int`

Nombre de points sur la face de devant

Par défaut: `1`

top `int`

Nombre de points sur la face du dessus

Par défaut: `2`

s `int` or `float` or `ratio`

Echelle du dessin

Par défaut: `1.0`

theta `int` or `float`

Angle theta des coordonnées sphériques

Par défaut: `70`

phi `int` or `float`

Angle phi des coordonnées sphériques

Par défaut: `110`

vue `str`

Vue « D » pour droite ou « G » pour gauche

Par défaut: `"D"`

color `color`

Couleur du dé, par défaut légèrement gris

Par défaut: `rgb("d3d3d3")`

dot-color `color`

Couleur des points

Par défaut: `black`

origin `array`

Position de l'origine du dé

Par défaut: `(0,0,0)`

box-args `dictionary`

Arguments supplémentaires pour la boîte

Par défaut: `()`

opacity `ratio`

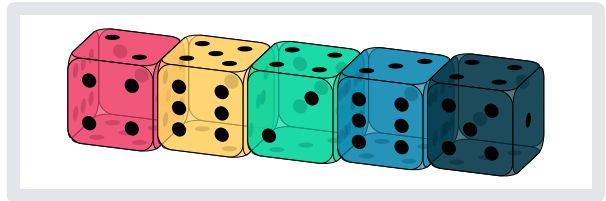
Opacité des dés

Par défaut: `100%`

throws

Tirage aléatoire ou non de n dés.

```
#throws(5, yams: true, colors:
(rgb("ef476f"), rgb("ffd166"),
rgb("06d6a0"), rgb("118ab2"),
rgb("073b4c")), seed: 999, s: 1.2, yams-
h: -.15, opacity: 70%,)
```



Paramètres

```
throws(
  n: int,
  list: none or array,
  seed: int,
  yams: boolean,
  yams-h: int float,
  espace-h: length,
  s: int float ratio,
  colors: array,
  dot-color: color,
  theta: int float,
  phi: int float,
  vue: str,
  box-args: dictionary,
  opacity: ratio
) -> content
```

n `int`

Nombre de dés

list `none` or array

Liste des jets `none` par défaut sinon liste des couples (face avant, face du dessus),

Par défaut: `none`

seed `int`

Graine du pseudo aléatoire

Par défaut: `42`

yams `boolean`

Dés présentés « collés »

Par défaut: `false`

yams-h `int` or `float`

Avancement des dés dans la ligne si yams

Par défaut: `.1`

espace-h `length`

Espacement horizontal des dés

Par défaut: `5mm`

s `int` or `float` or `ratio`

Echelle

Par défaut: `1.0`

colors `array`

Liste des couleurs des dés

Par défaut: `(rgb("d3d3d3"),)`

dot-color `color`

couleur des points

Par défaut: `black`

theta `int` or `float`

Angle theta des coordonnées sphériques

Par défaut: `70`

phi `int` or `float`

Angle phi des coordonnées sphériques

Par défaut: `110`

vue `str`

Vue « D » droite ou « G » gauche

Par défaut: `"D"`

box-args `dictionary`

Arguments supplémentaires pour la boîte

Par défaut: `()`

opacity `ratio`

Opacité des dés

Par défaut: `100%`

dice-face

Fonction créée par eric1102 sur Discord : dessine une face d'un dé avec la possibilité de changer la couleur du dé, des points et du trait

Paramètres

```
dice-face(  
  num: int,  
  fill: none color,  
  spot-fill: auto color,  
  stroke: auto stroke  
) -> content
```

num `int`

Entier de 1 à 6

fill `none` or `color`

Couleur du dé

Par défaut: `none`

spot-fill `auto` or `color`

Noir pour une couleur claire ou blanc pour une couleur sombre ou la couleur choisie

Par défaut: `auto`

stroke `auto` or `stroke`

Couleur du contour, si le dé est en couleur et stroke:auto alors même couleur

Par défaut: `auto`